

ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY AND SCIENCES

(UGC AUTONOMOUS)

**(Permanently Affiliated to AU, Approved by AICTE and Accredited by NBA
& NAAC with 'A+' Grade)**

SANGIVALASA, BHEEMILI MANDAL, VISAKHAPATNAM – 531162



BONAFIDE CERTIFICATE

This is to certify that the Mini Project entitled “**ACTIVITY 8: CONDITIONAL RANDOM FIELDS (CRF) IMPLEMENTATION**” submitted by **Team 8** in partial fulfilment of the requirements for the award of Bachelor of Technology in CSE (AI & ML) in the Department of Computer Science and Engineering (AI & ML), Anil Neerukonda Institute of Technology and Sciences (A+), Visakhapatnam, is a record of bonafide work carried out under my guidance and supervision.

B.Harshith Sai	A23126552199
Ch.GanaSurya	A23126552277
M.VenkatTeja	A23126552278
A.Chandu	A23126552279
K.kulwanth Hari Sai	A23126552280
K.L.S.Abhi Ram	A23126552281

Under the guidance of

Mrs.M. Krishnaveni

ASSISTANT PROFESSOR

Department of CSE (AI & ML)

Head of the Department

Prof. K. SELVANI DEEPTHI

PROFESSOR

Department of CSE (AI & ML)

ACTIVITY 8: CONDITIONAL RANDOM FIELDS (CRF) IMPLEMENTATION

1. OBJECTIVE

The main objective of this experiment is to implement a **Conditional Random Field (CRF)** model for performing sequence labeling tasks such as **Part-of-Speech (POS) tagging** and **Named Entity Recognition (NER)**.

Sequence labeling is an important task in Natural Language Processing (NLP), where each word in a sentence is assigned a label based on its role and context. Unlike traditional models, CRF considers the relationship between neighboring words, making it highly effective for structured prediction problems.

The specific objectives of this experiment are:

- To understand the working of Conditional Random Fields.
- To implement CRF using Python.
- To perform sequence labeling on sample text data.
- To evaluate the model performance using standard metrics.

2. INTRODUCTION

Natural Language Processing (NLP) is a field of Artificial Intelligence that focuses on enabling machines to understand and process human language. One of the fundamental tasks in NLP is **sequence labeling**, where each element in a sequence (such as words in a sentence) is assigned a corresponding label.

Common sequence labeling tasks include:

- Part-of-Speech (POS) tagging
- Named Entity Recognition (NER)
- Chunking

Traditional models such as Hidden Markov Models (HMM) and Maximum Entropy Markov Models (MEMM) have been used for sequence labeling. However, these models have limitations in handling dependencies between output labels.

Conditional Random Fields (CRF) overcome these limitations by modeling the conditional probability of the entire sequence of labels given the input sequence. CRFs are particularly effective because they consider the context of neighboring words, resulting in more accurate predictions.

3. SYSTEM OVERVIEW

The system developed in this experiment performs sequence labeling using a CRF model. It takes a sentence as input and outputs the corresponding labels for each word.

The system consists of the following components:

- Input processing module
- Feature extraction module
- CRF model training module
- Prediction module
- Evaluation module

Example:

Input: John lives in India

Output: NNP VBZ IN NNP

The CRF model learns from labeled training data and predicts labels for unseen sequences based on learned patterns.

4. DATASET

The dataset used in this experiment consists of sentences where each word is paired with a label. These labels represent the grammatical or semantic role of the word.

Example dataset:

```
[("John", "NNP"), ("loves", "VBZ"), ("India", "NNP")]  
[("She", "PRP"), ("is", "VBZ"), ("happy", "JJ")]
```

Each sentence is represented as a list of tuples:

- First element → Word
- Second element → Label

This structured format allows the CRF model to learn relationships between words and their corresponding labels.

5. DATA PROCESSING

The data processing phase involves preparing the input data for training the CRF model.

Step 1: Input Collection

Sentences are collected in the form of word-label pairs.

Step 2: Feature Extraction

For each word, a set of features is generated. These features help the model understand the characteristics of the word.

Features used:

- Lowercase form of word
- Whether the word is uppercase
- Whether the word is title case
- Previous word
- Next word

Step 3: Sequence Conversion

The dataset is converted into:

- $X \rightarrow$ Features
- $y \rightarrow$ Labels

Step 4: Train-Test Split

The dataset is divided into training and testing sets.

6. CRF MODEL

Conditional Random Fields are probabilistic models used for labeling sequential data. They model the conditional probability of label sequences given an input sequence.

CRF considers:

- Current word features
- Previous word features
- Transition between labels

Key advantages:

- Avoids label bias problem
- Considers entire sequence context
- Produces globally optimal predictions

7.CODE

```
!pip install sklearn-crfsuite

import nltk

import sklearn_crfsuite

from sklearn_crfsuite import metrics

nltk.download('treebank')

corpus = nltk.corpus.treebank.tagged_sents()

print(corpus)

# Define a function to extract features for each word in a sentence
def word_features(sentence, i):

    word = sentence[i][0]

    features = {

        'word': word,

        'is_first': i == 0, #if the word is a first word

        'is_last': i == len(sentence) - 1, #if the word is a last word

        'is_capitalized': word[0].upper() == word[0],

        'is_all_caps': word.upper() == word,    #word is in uppercase

        'is_all_lower': word.lower() == word,    #word is in lowercase

        #prefix of the word

        'prefix-1': word[0],

        'prefix-2': word[:2],

        'prefix-3': word[:3],

        #suffix of the word

        'suffix-1': word[-1],

        'suffix-2': word[-2:],

        'suffix-3': word[-3:],

        #extracting previous word

        'prev_word': " if i == 0 else sentence[i-1][0],

        #extracting next word
```

```

        'next_word': " if i == len(sentence)-1 else sentence[i+1][0],
        'has_hyphen': '-' in word,    #if word has hyphen
        'is_numeric': word.isdigit(), #if word is in numeric
        'capitals_inside': word[1:].lower() != word[1:]
    }

    return features

# Extract features for each sentence in the corpus
X = []
y = []

for sentence in corpus:
    X_sentence = []
    y_sentence = []
    for i in range(len(sentence)):
        X_sentence.append(word_features(sentence, i))
        y_sentence.append(sentence[i][1])
    X.append(X_sentence)
    y.append(y_sentence)

# Split the data into training and testing sets
split = int(0.8 * len(X))
X_train = X[:split]
y_train = y[:split]
X_test = X[split:]
y_test = y[split:]

# Train a CRF model on the training data
crf = sklearn_crfsuite.CRF(
    algorithm='lbfgs',
    c1=0.1,
    c2=0.1,

```

```

        max_iterations=100,
        all_possible_transitions=True
    )
    crf.fit(X_train, y_train)

# Make predictions on the test data and evaluate the performance
y_pred = crf.predict(X_test)

print(metrics.flat_accuracy_score(y_test, y_pred))

import pycrfsuite

# Train a CRF model suing pysrfsuite
trainer = pycrfsuite.Trainer(verbose=False)
for x, y in zip(X_train, y_train):
    trainer.append(x, y)
trainer.set_params({
    'c1': 1.0,
    'c2': 1e-3,
    'max_iterations': 50,
    'feature.possible_transitions': True
})
trainer.train('pos.crfsuite')

# Tag a new sentence
tagger = pycrfsuite.Tagger()
tagger.open('pos.crfsuite')
sentence = list(map(str,input().split()))
features = [word_features(sentence, i) for i in range(len(sentence))]
tags = tagger.tag(features)print(list(zip(sentence, tags)))

```

7. IMPLEMENTATION

Step 1: Install Required Libraries

pip install sklearn-crfsuite

```
❏ pip install sklearn-crfsuite

... Collecting sklearn-crfsuite
  Downloading sklearn_crfsuite-0.5.0-py2.py3-none-any.whl.metadata (4.9 kB)
Collecting python-crfsuite>=0.9.7 (from sklearn-crfsuite)
  Downloading python_crfsuite-0.9.12-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.metadata (4.3 kB)
Requirement already satisfied: scikit-learn>=0.24.0 in /usr/local/lib/python3.12/dist-packages (from sklearn-crfsuite) (1.6.1)
Requirement already satisfied: tabulate>=0.4.2 in /usr/local/lib/python3.12/dist-packages (from sklearn-crfsuite) (0.9.0)
Requirement already satisfied: tqdm>=2.0 in /usr/local/lib/python3.12/dist-packages (from sklearn-crfsuite) (4.67.3)
Requirement already satisfied: numpy>=1.19.5 in /usr/local/lib/python3.12/dist-packages (from scikit-learn>=0.24.0->sklearn-crfsuite) (2.0.2)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn>=0.24.0->sklearn-crfsuite) (1.16.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn>=0.24.0->sklearn-crfsuite) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn>=0.24.0->sklearn-crfsuite) (3.6.0)
Downloading sklearn_crfsuite-0.5.0-py2.py3-none-any.whl (10 kB)
Downloading python_crfsuite-0.9.12-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl (1.2 MB)
1.2/1.2 MB 26.3 MB/s eta 0:00:00
Installing collected packages: python-crfsuite, sklearn-crfsuite
Successfully installed python-crfsuite-0.9.12 sklearn-crfsuite-0.5.0
```

Step 2: Import Libraries

```
import sklearn_crfsuite
from sklearn_crfsuite import metrics
```

```
❏ import nltk
import sklearn_crfsuite
from sklearn_crfsuite import metrics
```

```
❏ nltk.download('treebank')
corpus = nltk.corpus.treebank.tagged_sents()
print(corpus)

... [nltk_data] Downloading package treebank to /root/nltk_data...
[nltk_data] Unzipping corpora/treebank.zip.
[[('Pierre', 'NNP'), ('Vinken', 'NNP'), ('.', ','), ('61', 'CD'), ('years', 'NNS'), ('old', 'JJ'),
```

Step 3: Define Feature Function

```
def word2features(sent, i):
    word = sent[i][0]

    features = {
        'word.lower()': word.lower(),
        'word.isupper()': word.isupper(),
        'word.istitle()': word.istitle(),
    }

    if i > 0:
        features['-1:word'] = sent[i-1][0]
    else:
        features['BOS'] = True

    if i < len(sent)-1:
```

```

    features['+1:word'] = sent[i+1][0]
else:
    features['EOS'] = True

```

```

return features

```

▶ # Define a function to extract features for each word in a sentence

```

def word_features(sentence, i):
    word = sentence[i][0]
    features = {
        'word': word,
        'is_first': i == 0, #if the word is a first word
        'is_last': i == len(sentence) - 1, #if the word is a last word
        'is_capitalized': word[0].upper() == word[0],
        'is_all_caps': word.upper() == word, #word is in uppercase
        'is_all_lower': word.lower() == word, #word is in lowercase
        #prefix of the word
        'prefix-1': word[0],
        'prefix-2': word[:2],
        'prefix-3': word[:3],
        #suffix of the word
        'suffix-1': word[-1],
        'suffix-2': word[-2:],
        'suffix-3': word[-3:],
        #extracting previous word
        'prev_word': '' if i == 0 else sentence[i-1][0],
        #extracting next word
        'next_word': '' if i == len(sentence)-1 else sentence[i+1][0],
        'has_hyphen': '-' in word, #if word has hyphen
        'is_numeric': word.isdigit(), #if word is in numeric
        'capitals_inside': word[1:].lower() != word[1:]
    }
    return features

```

Step 4: Prepare Data

```

X = [[word2features(s, i) for i in range(len(s))] for s in sentences]

```

```

y = [[label for (word, label) in s] for s in sentences]

```

```
# Extract features for each sentence in the corpus
X = []
y = []
for sentence in corpus:
    X_sentence = []
    y_sentence = []
    for i in range(len(sentence)):
        X_sentence.append(word_features(sentence, i))
        y_sentence.append(sentence[i][1])
    X.append(X_sentence)
    y.append(y_sentence)

# Split the data into training and testing sets
split = int(0.8 * len(X))
X_train = X[:split]
y_train = y[:split]
X_test = X[split:]
y_test = y[split:]
```

Step 5: Train Model

```
crf = sklearn_crfsuite.CRF()
crf.fit(X, y)
```

```
 # Train a CRF model on the training data
crf = sklearn_crfsuite.CRF(
    algorithm='lbfgs',
    c1=0.1,
    c2=0.1,
    max_iterations=100,
    all_possible_transitions=True
)
crf.fit(X_train, y_train)

# Make predictions on the test data and evaluate the performance
y_pred = crf.predict(X_test)

print(metrics.flat_accuracy_score(y_test, y_pred))
```

```
... 0.9632716203403363
```

Step 6: Prediction

```
y_pred = crf.predict(X)
```

```

▶ import pycrfsuite

# Train a CRF model using pycrfsuite
trainer = pycrfsuite.Trainer(verbose=False)
for x, y in zip(X_train, y_train):
    trainer.append(x, y)
trainer.set_params({
    'c1': 1.0,
    'c2': 1e-3,
    'max_iterations': 50,
    'feature.possible_transitions': True
})
trainer.train('pos.crfsuite')

# Tag a new sentence
tagger = pycrfsuite.Tagger()
tagger.open('pos.crfsuite')
sentence = list(map(str, input().split()))
features = [word_features(sentence, i) for i in range(len(sentence))]
tags = tagger.tag(features)
print(list(zip(sentence, tags)))

... chandu is aiml student
[('chandu', 'NN'), ('is', 'NNS'), ('aiml', 'DT'), ('student', 'NNS')]

```

Step 7: Evaluation

```
print(metrics.flat_classification_report(y, y_pred))
```

8. RESULTS

The CRF model successfully predicted labels for the given sentences.

Sample Output:

Input: John loves India

Output: NNP VBZ NNP

...

```

... john loves india
[('john', 'NN'), ('loves', 'NN'), ('india', 'NNS')]

```

Observations:

- Model performs well for simple sentences
- Context improves prediction accuracy

- Neighboring words influence labeling

9. EVALUATION

The model is evaluated using the following metrics:

- Accuracy
- Precision
- Recall
- F1-score

These metrics help in understanding the performance of the model.

High accuracy indicates that the CRF model is effective in sequence labeling tasks.

10. DISCUSSION

Strengths:

- Captures context effectively
- Produces accurate predictions
- Suitable for structured data

Challenges:

- Requires feature engineering
- Performance depends on dataset quality

11. LIMITATIONS

- Limited dataset reduces accuracy
- Requires manual feature design
- Not suitable for very large datasets without optimization

12. FUTURE WORK

- Use deep learning models like BiLSTM-CRF
- Train on larger datasets

- Apply to real-world NER applications
- Improve feature engineering techniques

13. CONCLUSION

In this experiment, a Conditional Random Field model was implemented for sequence labeling tasks. The model successfully learned patterns from the dataset and provided accurate predictions.

CRF proved to be an effective approach for tasks such as POS tagging and Named Entity Recognition due to its ability to consider contextual information.

This experiment demonstrates the importance of structured prediction models in Natural Language Processing.

14. REFERENCES

- sklearn-crfsuite documentation
- Python official documentation
- Jurafsky & Martin – Speech and Language Processing
- NLP tutorials and research papers